2. One-Shot Build Instructions for the Coding Agent

2.1 Build Objective

The coding agent shall build a production-ready **web-based Street Names Registration and Management System (SNRMS)** for **Ibeju-Lekki Local Government Area**.

The system shall enable applicants to:

- verify proposed street-name availability,
- submit new street-name registration applications,
- renew approved street-name registrations,
- upload all required supporting documents,
- make required stage-based payments,
- track application progress across all review and operational stages, and
- receive application-related notifications and status updates.

The system shall also provide role-based operational interfaces for internal users to:

- review submitted applications,
- validate documents,
- confirm payments,
- make approval or rejection decisions,
- update downstream operational stages, and
- manage the official street-name registration lifecycle.

The solution shall be implemented as a **responsive web application with mobile-first design**, optimized for small-screen devices first and scaling cleanly to tablet and desktop screen sizes without loss of functionality.

2.2 Required Output Artifacts

The generated solution shall include all of the following:

2.2.1 Frontend

- A fully functional **Vue.js frontend application**
- Responsive user-facing and staff-facing interfaces
- Role-aware navigation and dashboards

2.2.2 Backend

- A fully functional **Django backend application**
- RESTful API endpoints for all required workflows
- Business-rule enforcement on the server side

2.2.3 Database

- A **PostgreSQL** database schema
- Database migrations for all domain entities
- Seed data for local development and testing

2.2.4 Authentication and Authorization

- Role-based authentication and authorization
- Applicant self-registration
- Staff account creation restricted to Committee Chairman users
- Password login and password recovery support

2.2.5 Applicant Workflows

The system shall support applicant-facing workflows for:

- street-name confirmation,
- new street-name registration,
- street-name renewal,
- application tracking,
- notification viewing,
- document upload,
- Stage A payment submission, and
- Stage C final payment submission.

2.2.6 Internal Staff Workflows

The system shall support staff-facing dashboards and workflows for:

- Finance,
- Naming Committee, and
- Committee Chairman.

2.2.7 Operational Capabilities

The system shall support:

- payment confirmation,
- document validation,
- recommendation, approval, and rejection actions,
- post-approval operational stage updates,
- application status tracking across all required stages,
- notification and event history visible to applicants, and

- audit-friendly activity records for key workflow actions.

2.2.8 Quality and Delivery Artifacts

The generated solution shall include:

- automated tests for core backend and frontend flows,
- local setup instructions in a README,
- Docker or docker-compose based local development support, and
- API documentation or clearly structured endpoint definitions within the codebase.

2.3 Technology Stack Constraints

The coding agent shall use the following technology stack and shall not replace it unless explicitly instructed.

2.3.1 Frontend Technology

The frontend shall be implemented using:

- **Vue.js**
- Preferably **Vue 3**
- **Vue Router** for client-side routing
- **Pinia** or an equivalent maintainable Vue state-management pattern

The frontend shall:

- follow a mobile-first responsive design approach,
- render correctly on narrow mobile screens,
- scale cleanly to tablet and desktop layouts, and
- remain fully usable across supported screen sizes.

2.3.2 Backend Technology

The backend shall be implemented using:

- **Django**
- **Django REST Framework** for API development

The backend shall enforce:

- role-based permissions,
- server-side validation for all critical business actions,
- application workflow rules,
- payment confirmation rules, and
- document validation rules.

2.3.3 Database Technology

The system database shall use:

- **PostgreSQL**

2.3.4 File Storage

Uploaded files shall:

- be handled through Django's file handling system,
- be linked to the correct application record,
- support local development storage, and
- be implemented in a way that can later support cloud migration without major redesign.

2.3.5 API Style

The application programming interface shall use:

- **REST API** only

2.3.6 Authentication Model

Authentication shall be Django-based and role-aware.
The implementation may use session-based or token-based authentication, provided that:

- the approach is secure,
- frontend and backend behavior remain consistent,
- role-based access is enforced on the backend, and
- password recovery is supported.

2.4 Mandatory User Roles

The system shall use exactly the following four user roles:

1. **Applicant**
2. **Finance**
3. **Naming Committee**
4. **Committee Chairman**

No additional user role shall be introduced unless explicitly specified in a later revision.

The earlier split between **Committee Chairman** and **Executive Chairman** is removed. Any action previously intended for the Executive Chairman shall now be performed by the **Committee Chairman** role.

Any user who self-registers through the public registration flow shall be created as an **Applicant** by default.

Only the **Committee Chairman** shall be permitted to create internal staff accounts for:

- Finance,
- Naming Committee, and
- other Committee Chairman users if required by future policy.

2.5 Core Product Capabilities to Implement

The coding agent shall implement the following capabilities.

2.5.1 Applicant Capabilities

Applicants shall be able to:

- create an account,
- sign in,
- recover their password,
- view an applicant dashboard,
- confirm whether a proposed street name is available,
- start a new street-name registration application,
- upload all required application documents,
- submit Stage A payment information,
- track application progress by stage and status,
- receive application updates and notifications,
- submit Stage C final payment information after approval,
- renew an existing approved street-name registration, and
- view renewal status.

2.5.2 Finance Capabilities

Finance users shall be able to:

- view submitted payment records,
- confirm Stage A payments,
- confirm Stage C final payments,
- view applicant details, application references, payment type, amount, and payment status,
- trigger next-stage progression after successful payment confirmation, and
- configure payment amounts for required payment categories and stages, subject to business rules.

2.5.3 Naming Committee Capabilities

Naming Committee users shall be able to:

- review uploaded recognition letters from the Oba, Baale, or Royal Father,
- validate or reject the royal recognition document,
- record validation outcomes,

- update downstream operational stages after approval, including post-payment execution stages, and
- move an application forward only where required document conditions are satisfied.

2.5.4 Committee Chairman Capabilities

Committee Chairman users shall be able to:

- review screened applications,
- inspect submitted documents and prior validations,
- recommend, approve, or reject applications in accordance with the merged approval workflow,
- trigger applicant notification for final payment where applicable,
- monitor overall pipeline status and stage counts,
- view dashboard summaries of financial status and application counts, and
- create internal staff accounts for Finance, Naming Committee, and Committee Chairman users.

2.6 Workflow Scope to Implement

The system shall support the following business workflows.

2.6.1 Flow A — Street Name Confirmation

An Applicant shall be able to search for a proposed street name and receive a result indicating whether the street name is:

- available,
- already registered,
- already under application, or
- otherwise not allowed.

2.6.2 Flow B — New Registration

An Applicant shall be able to submit a new street-name registration through a multi-step form, upload all required documents, submit Stage A payment information, and enter the internal review pipeline.

2.6.3 Flow C — Stage A Finance Confirmation

A Finance user shall confirm the initial payment before the application can proceed to the next review stage.

2.6.4 Flow D — Naming Committee Validation

A Naming Committee user shall validate the Royal Father recognition letter and related document conditions, and either approve or reject that review stage.

2.6.5 Flow E — Committee Screening and Decision

A Committee Chairman user shall review the application after payment confirmation and Naming Committee validation, and shall either:

- reject the application, or
- approve it for Stage C final payment.

2.6.6 Flow F — Stage C Final Payment

After approval, the Applicant shall be able to submit Stage C final payment information. A Finance user shall confirm the final payment before the application proceeds to operational completion stages.

2.6.7 Flow G — Post-Payment Completion

After Stage C confirmation, the application shall progress through downstream operational states such as:

- signpost installation,
- map update,
- certificate readiness, and
- final completion.

The Naming Committee shall be able to update these downstream states so that the Applicant can see operational progress.

2.6.8 Flow H — Renewal

An Applicant shall be able to select an existing approved street-name registration and submit a renewal request tied to that approved record.

2.7 Required Stage Model

The application lifecycle shall be implemented as explicit tracked stages.

At minimum, the system shall support the following stages:

- **Stage A — Initial payment and submission**
- **Stage B — Document review and Naming Committee validation**
- **Stage C — Final approval and final payment**
- **Stage D — Signpost installation**
- **Stage E — Map update**
- **Stage F — Certificate and completion**

Each application shall always have:

- a clearly stored current stage, and
- a clearly stored current status.

Stage and status shall be stored separately so that the system can represent both process phase and process outcome.

2.8 Required Application Status Behavior

Each application shall support explicit, machine-readable status values.

At minimum, the backend shall support statuses covering:

- `draft`
- `submitted`
- `awaiting_stage_a_payment`
- `stage_a_paid`
- `stage_a_confirmed`
- `under_naming_committee_review`
- `naming_committee_rejected`
- `under_committee_review`
- `committee_rejected`
- `approved_awaiting_stage_c_payment`
- `stage_c_paid`
- `stage_c_confirmed`
- `signpost_pending`
- `maps_update_pending`
- `certificate_ready`
- `completed`
- `renewal_pending`
- `renewed`
- `cancelled`

The exact internal names may be standardized later, but the workflow logic shall preserve these distinct states.

The system shall enforce valid state transitions and shall prevent unauthorized or logically invalid progression.

2.9 Required Applicant Documents

The registration workflow shall support and require at minimum the following applicant documents:

- NIN verification slip
- Passport photograph
- Royal Father recognition letter
- Survey or property documents

- Completed application form

All listed documents shall be treated as mandatory unless a later approved revision changes that rule.

Workflow progression shall be blocked if:

- any mandatory document is missing,
- any mandatory document is invalid,
- any mandatory document is not yet reviewed where review is required.

The Royal Father recognition letter shall require explicit Naming Committee validation before the application can progress to the relevant next decision stage.

2.10 Fee and Street-Type Awareness

The system shall support street-type-aware pricing and application rules.

At minimum, the system shall recognize the following street types:

- Court
- Crescent
- Way
- Close
- Street
- Road
- Avenue
- Boulevard
- Lane
- Drive
- Parkway
- Mews
- Esplanade
- Circus
- Terrace
- Plaza / Mall

Street-type definitions and fees shall be stored as configurable data and shall not be hard-coded only in the frontend.

The coding agent shall model street-type-related master data with at least the following attributes:

- street type name,
- description,
- abbreviation,
- fee amount,
- active or inactive flag.

The implementation shall allow fee values to be updated without requiring frontend code changes.

2.11 Required Payment Support

The system shall support at minimum the following payment categories.

2.11.1 Stage A Payment

Stage A payment shall support the following fee components:

- application fee,
- inspection fee,
- radio and TV tax, and
- committee verification fee.

2.11.2 Stage C Final Payment

Stage C payment shall support the following fee components:

- street-name fee based on selected street type,
- signpost installation fee, and
- map upload fee.

2.11.3 Payment Data Requirements

For each payment record, the system shall store at minimum:

- payment reference,
- application reference,
- payer,
- payment stage,
- amount,
- payment status,
- confirmation status,
- confirmed by,
- confirmation timestamp.

2.11.4 Payment Confirmation Model

The system shall support manual confirmation by Finance even if an online payment gateway is added later.

The version 1 design shall not assume fully automated payment reconciliation unless explicitly specified in a later revision.

2.12 Non-Negotiable Implementation Rules

The coding agent shall follow these rules strictly:

- The solution shall be implemented as a **web application**, not a mobile-native application.
- The interface shall be designed **mobile-first** and scale cleanly to desktop.
- No extra user roles shall be introduced.
- Committee Chairman and Executive Chairman shall not be split again.
- Vue.js, Django, and PostgreSQL shall not be replaced.
- Mock endpoints shall not appear in the final output.
- TODO placeholders shall not remain in production code.
- Validation shall not be skipped for any financial, approval, account-creation, or status-transition action.
- Critical business rules shall be enforced on the backend and shall not exist only in the frontend.
- All file uploads shall be linked to the correct application.
- All state transitions shall be recorded consistently.
- The interface shall remain usable on low-width mobile screens.

2.13 Preferred Implementation Order

The coding agent should implement the solution in the following order:

1. Define domain models and database schema
2. Create migrations
3. Seed master and reference data, including:
   - user roles,
   - street types,
   - fee categories,
   - application statuses
4. Implement authentication and role management
5. Implement core application workflow models
6. Implement the document upload subsystem
7. Implement payment recording and payment-confirmation flows
8. Implement role-based staff dashboards
9. Implement applicant-facing dashboards and multi-step forms
10. Implement tracking and notification history
11. Implement renewal flow
12. Add validations and business rules
13. Add automated tests
14. Add Docker or local container setup
15. Finalize README and local setup guidance

2.14 Forbidden Assumptions

The coding agent shall not assume any of the following unless explicitly specified later:

- that online payment is fully automated end to end,
- that Paystack is the final selected payment provider,
- that direct Google Maps API integration is required in version 1,
- that certificate generation must produce downloadable PDFs in version 1,
- that GIS visualization is required in version 1,
- that public users may modify approved records,
- that rejected applications should be auto-deleted,
- that file storage is cloud-only,
- that street names become active immediately after applicant submission.

2.15 Minimum Quality Bar for Completion

The project shall not be considered complete unless all of the following are true:

- applicant registration works end to end,
- applicant login and password recovery work,
- role-based login works,
- an Applicant can create and track an application,
- Finance can confirm Stage A and Stage C payments,
- Naming Committee can validate or reject royal recognition letters,
- Committee Chairman can review and approve or reject applications,
- Committee Chairman can create internal staff accounts,
- stage transitions are persisted correctly,
- uploaded documents are accessible to authorized internal reviewers,
- frontend behavior is responsive across mobile and desktop layouts,
- core backend tests pass,
- core frontend flows render and behave correctly,
- the project runs locally with documented setup steps.

2.16 Output Style Expected from the Coding Agent

The generated solution shall be:

- complete,
- explicit,
- maintainable,
- modular,
- readable,
- strongly validated, and
- ready for iterative enhancement.

The coding agent should prefer straightforward, dependable, and easily understandable implementation over unnecessary abstraction or cleverness.

This section is now strong enough to lock.

One thing I would still recommend for the next section is that we keep one naming convention consistently everywhere:

- either **Street Names Registration and Management System**
- or **Street Naming Registration and Management System**

Right now, **Street Names Registration and Management System (SNRMS)** is acceptable, but we should keep exactly one official expansion across the full document.

Next, we should move to **Section 4 — System Context**.

4. System Context

4.1 System Overview

The **Street Names Registration and Management System (SNRMS)** shall be a web-based platform used to manage the end-to-end process of street-name verification, registration, approval, payment confirmation, renewal, and post-approval operational tracking within **Ibeju-Lekki Local Government Area**.

The system shall serve two broad classes of users:

- **external users**, namely Applicants who initiate and track street-name requests; and
- **internal users**, namely Finance, Naming Committee, and Committee Chairman users who review, validate, approve, and operationally update applications.

The system shall act as the central digital workflow system for street-name application processing and shall replace fragmented, manual, or partially paper-based handling of the process.

4.2 Business Purpose

The purpose of the system is to provide a controlled, traceable, and role-based process for managing official street-name registration activities.

The system shall support the following business goals:

- improve transparency in the street-name registration lifecycle,

- reduce ambiguity in application handling,
- provide applicants with visibility into application progress,
- enforce required document and payment checks before approval,
- centralize application records and operational decisions,
- support staff accountability through role-based workflows and activity history, and
- enable structured renewal and post-approval tracking.

4.3 System Boundary

The system boundary shall include all functionality required to:

- allow Applicants to search and confirm proposed street names,
- receive and store new registration applications,
- receive and store renewal applications,
- receive and manage uploaded applicant documents,
- record required payment submissions,
- allow Finance to confirm payments,
- allow Naming Committee to validate required recognition documents and update downstream operational stages,
- allow Committee Chairman to review applications and make final approval or rejection decisions,
- notify Applicants of important application events and status changes,
- track current stage and status for every application, and
- maintain master records for configurable street types and fee settings.

The system boundary shall exclude, unless later approved, the following:

- direct GIS visualization,
- direct public map publishing,
- fully automated external payment gateway reconciliation,
- downloadable certificate generation as a mandatory first-version feature,
- public self-service editing of approved street records,
- physical signpost execution management beyond status tracking,
- advanced analytics beyond dashboard summaries,
- native Android or iOS mobile applications.

4.4 Primary Actors

The system shall recognize the following primary actors.

4.4.1 Applicant

The Applicant is an external user who interacts with the public-facing parts of the system to:

- create an account,
- sign in,
- recover password,
- confirm street-name availability,
- submit new applications,
- upload documents,
- submit payment information,
- track status and stage progression,
- view notifications, and
- request renewal of approved records.

4.4.2 Finance

The Finance user is an internal operational actor responsible for:

- viewing payment submissions,
- confirming Stage A payments,
- confirming Stage C final payments,
- reviewing payment details, and
- enabling progression after valid payment confirmation.

4.4.3 Naming Committee

The Naming Committee user is an internal operational actor responsible for:

- reviewing recognition letters and related document conditions,
- validating or rejecting required recognition evidence,
- updating downstream operational states after approval, and
- ensuring that required non-financial review conditions are satisfied before progression.

4.4.4 Committee Chairman

The Committee Chairman is the highest workflow authority within the defined role model and is responsible for:

- reviewing screened applications,
- making recommendation, approval, or rejection decisions,
- viewing pipeline and financial summaries,
- triggering applicant progression to final payment where applicable, and
- creating internal staff accounts for Finance, Naming Committee, and Committee Chairman users.

4.5 Secondary Actors and External Dependencies

The system may interact with or depend on the following secondary actors or external service categories.

4.5.1 Email Service

The system may use an email service for account recovery, account notifications, and workflow event communication.

4.5.2 File Storage Layer

The system shall depend on a file storage mechanism for applicant-uploaded documents. In local development this shall use Django-supported local storage, with design compatibility for future cloud migration.

4.5.3 Payment Reference Source

The system may store payment references generated from manual or external payment processes. The first version shall support manual confirmation and shall not depend on a specific payment gateway being fully integrated.

4.5.4 Administrative Reference Data Maintainers

Internal authorized users, especially Committee Chairman and permitted operational users, may maintain reference data such as fee values and street-type definitions, subject to detailed rules defined later in the specification.

4.6 Operational Environment

The system shall operate as a browser-based web application.

4.6.1 Client Environment

The frontend shall be accessible through modern web browsers on:

- mobile phones,
- tablets, and
- desktop or laptop computers.

The interface shall follow a mobile-first design approach and remain fully usable on narrow screens before scaling up to larger layouts.

4.6.2 Server Environment

The backend shall run as a Django application connected to a PostgreSQL database and shall expose REST API endpoints for frontend and internal workflow operations.

4.6.3 Deployment Environment

The system shall support local containerized development through Docker or docker-compose. Production hosting details may be defined later, but the architecture shall assume standard web deployment patterns with separated frontend, backend, database, and media storage concerns.

4.7 Core Business Objects in Context

The system context shall revolve around the following core business objects:

- **User**
- **Role**
- **Application**
- **Street Name Request**
- **Renewal Request**
- **Street Type**
- **Payment**
- **Document**
- **Notification**
- **Status History / Activity Log**
- **Operational Stage Update**

These objects represent the minimum contextual entities required for the workflow and shall later be defined formally in the data and database sections.

4.8 High-Level Interaction Model

The expected high-level interaction model shall be as follows:

1. An Applicant accesses the public web interface.
2. The Applicant confirms whether a proposed street name is available.
3. The Applicant submits a new application or renewal request.
4. The Applicant uploads required documents.
5. The Applicant submits Stage A payment information.
6. Finance confirms payment.
7. Naming Committee validates required recognition evidence.
8. Committee Chairman reviews the screened application and either rejects it or approves it for Stage C final payment.
9. The Applicant submits Stage C payment information.
10. Finance confirms the final payment.
11. Naming Committee updates downstream operational progress such as signpost, map update, and certificate readiness.
12. The Applicant tracks progress and receives notifications until completion.

This interaction model shall be treated as the baseline process context for the system.

4.9 Contextual Inputs

The system shall accept the following major categories of inputs:

- applicant profile data,
- login credentials,
- proposed street-name search input,
- registration form data,
- renewal request data,
- uploaded documents,
- payment records and references,
- internal validation outcomes,
- internal approval or rejection decisions,
- operational stage updates,
- fee and street-type configuration data.

4.10 Contextual Outputs

The system shall produce the following major categories of outputs:

- street-name availability results,
- application confirmation records,
- stage and status updates,
- payment confirmation states,
- validation outcomes,
- approval or rejection outcomes,
- applicant-visible notifications,
- internal operational dashboards,
- financial summaries,
- pipeline and stage counts,
- application activity history.

4.11 Authority and Decision Flow Context

The system shall reflect the following authority structure:

- Applicants initiate and monitor requests but do not approve or validate them.
- Finance confirms required payments.
- Naming Committee validates the required recognition document and updates downstream operational stages.
- Committee Chairman performs the final review authority within the defined four-role model and controls internal staff account creation.

No role below Committee Chairman shall be able to create privileged internal staff accounts unless a future revision explicitly changes this rule.

4.12 Traceability Expectations in Context

Because the system manages official workflow decisions, it shall support operational traceability.

At minimum, the system context assumes that important actions shall be attributable to a user and timestamp, including:

- application submission,
- payment submission,
- payment confirmation,
- document validation,
- approval or rejection,
- staff account creation,
- status changes,
- stage changes,
- renewal submission, and
- downstream operational updates.

This traceability expectation shall influence later API, data, and acceptance specifications.

4.13 Context Assumptions

The system context is based on the following assumptions:

- Applicants can self-register through the public interface.
- Internal staff users are created through controlled internal workflows.
- Street-name processing is multi-stage and role-dependent.
- Payment confirmation may initially be manual.
- Required documents must be present before approval progression.
- Post-approval operational steps are tracked in the system even where the real-world execution occurs outside the software.
- The first version is a responsive web application, not a native mobile application.

4.14 Context Constraints

The system shall operate under the following constraints:

- exactly four user roles shall be used,
- role behavior shall follow the defined workflow chain,
- the application must be usable on mobile and desktop browsers,
- business-critical rules shall be enforced on the backend,

- workflow progression shall be explicit and tracked,
- version 1 shall not depend on mandatory GIS integration,
- version 1 shall not depend on mandatory automated payment gateway integration,
- version 1 shall not depend on mandatory certificate PDF generation.

4.15 Context Summary

In summary, the SNRMS shall function as a controlled, role-based, browser-accessible workflow platform for official street-name registration and renewal processing in Ibeju-Lekki LGA. It shall connect Applicants, Finance, Naming Committee, and Committee Chairman users through a structured approval and operational pipeline, while maintaining visibility, traceability, and configurable domain data such as street types and fee structures.

6. User Roles and Permissions

6.1 Overview

The system shall implement **strict role-based access control (RBAC)**.
Every authenticated user shall be assigned exactly one primary role, and all system actions shall be governed by role-based permissions enforced on the backend.

No user shall be allowed to perform actions outside the permissions explicitly defined for their role.

The system shall support exactly four roles:

1. Applicant
2. Finance
3. Naming Committee
4. Committee Chairman

6.2 Role Assignment Rules

6.2.1 Applicant Role Assignment

- Any user who registers through the public interface shall automatically be assigned the **Applicant** role.
- Applicants shall not be able to elevate their role.

6.2.2 Staff Role Assignment

- Only the **Committee Chairman** shall be authorized to create internal staff accounts.
- Staff roles include:
  - Finance
  - Naming Committee
  - Committee Chairman

6.2.3 Role Immutability

- A user’s role shall not be changeable by the user themselves.
- Role changes (if allowed in the future) must be performed only by the **Committee Chairman**.

6.3 Permission Model

Permissions shall be enforced at the **backend level** using Django role-based access control.

Permissions shall be defined at the level of:

- resource access (e.g., application, payment, document),
- action (e.g., view, create, update, approve),
- and scope (e.g., own application vs all applications).

6.4 Resource Ownership Rules

- An **Applicant** owns only their own applications and related data.
- Internal users (Finance, Naming Committee, Committee Chairman) shall have visibility across applications according to their role.
- No Applicant shall access another Applicant's data.

6.5 Permission Matrix

The following defines the minimum required permissions.

6.5.1 Application Management

Action	Applicant	Finance	Naming Committee	Committee Chairman
e application				
own application				
all applications				
raft application				
t application				

l own application

application

ve application

6.5.2 Payment Management

Action	Applicant	Finance	Naming Committee	Committee Chairman
t payment info				
own payments				
ll payments				
m Stage A payment				
m Stage C payment				
ure payment amounts				

6.5.3 Document Management

Action	Applicant	Finance	Naming Committee	Committee Chairman

d documents

own documents

ll documents

te recognition letter

recognition letter

6.5.4 Workflow and Status Control

Action	Applicant	Finance	Naming Committe	Committee Chairma
application status				
e Stage A confirmation				
e Stage B validation				
e downstream stages (D–F)				
application to approval stage				
approval decision				

6.5.5 User Management

Action	Applicant	Finance	Naming Committee	Committee Chairman
egister				
own profile				
e staff account				
all users				
n roles				

6.5.6 Notifications and Tracking

Action	Applicant	Finance	Naming Committee	Committee Chairman
own notifications				
system activity logs				

6.6 Access Scope Rules

6.6.1 Applicant Scope

Applicants shall:

- only access their own data,
- only modify applications in allowed states (e.g., draft),

- not access internal decision or validation controls.

6.6.2 Finance Scope

Finance users shall:

- access all applications for payment visibility,
- not modify application content,
- not approve or reject applications.

6.6.3 Naming Committee Scope

Naming Committee users shall:

- access all applications,
- validate recognition documents,
- update downstream operational stages,
- not confirm payments,
- not perform final approval decisions.

6.6.4 Committee Chairman Scope

Committee Chairman users shall:

- access all system data,
- perform final approval and rejection,
- create internal staff accounts,
- view financial summaries,
- override workflow progression where required.

6.7 Permission Enforcement Rules

The system shall enforce permissions as follows:

- All permissions must be enforced on the backend using Django.
- The frontend must not be relied upon for enforcing security.
- Unauthorized actions must return appropriate HTTP error responses:
  - 401 for unauthenticated access,
  - 403 for unauthorized actions.
- Every sensitive action must validate:

- user role,
- application state,
- required prerequisites (e.g., payment confirmed before progression).

6.8 Role-Based UI Behavior

The frontend shall adapt based on role:

- Applicants shall see only applicant dashboards and actions.
- Finance shall see payment dashboards.
- Naming Committee shall see document validation and stage update dashboards.
- Committee Chairman shall see full system dashboards and controls.

UI elements shall not expose unauthorized actions even if backend enforcement exists.

6.9 Separation of Duties

The system shall enforce separation of responsibilities:

- Payment confirmation (Finance) must be independent of approval (Committee Chairman).
- Document validation (Naming Committee) must be independent of financial confirmation.
- Final approval authority must remain exclusively with Committee Chairman.

This separation is required for accountability and audit integrity.

6.10 Audit Responsibility

All actions performed by users must be attributable to:

- a user ID,
- a role,
- a timestamp.

This includes:

- approvals,
- rejections,
- payment confirmations,
- document validations,
- staff account creation,
- stage updates.

6.11 Constraints

- No role escalation shall be possible through the UI.
- No internal role shall be assignable through public registration.
- No user shall bypass required workflow steps through direct API calls.
- All permissions must be consistently enforced across all endpoints.

6.12 Summary

The role and permission model defines a strict workflow hierarchy:

- Applicants initiate and track requests.
- Finance confirms payments.
- Naming Committee validates documents and updates operational stages.
- Committee Chairman performs final approvals and manages internal users.

This model ensures:

- clear separation of duties,
- accountability,
- controlled progression through stages, and
- secure system operation.

7. User Journeys and Core Workflows

7.1 Overview

This section defines the **end-to-end user journeys and system workflows** that the application must support.

Each workflow shall specify:

- triggering actor,
- required inputs,
- ordered steps,
- system actions,
- success outcomes, and
- failure or rejection paths.

All workflows defined here must be fully implementable without ambiguity.

7.2 Primary Workflow: Street Name Confirmation

7.2.1 Actor

Applicant

7.2.2 Trigger

Applicant wants to check if a proposed street name is available before starting an application.

7.2.3 Steps

1. Applicant navigates to “Confirm Street Name” page.
2. Applicant enters proposed street name.
3. Applicant submits the query.
4. System normalizes input (case-insensitive comparison).
5. System searches:
   - existing approved street names,
   - existing pending applications.
6. System returns result.

7.2.4 Outcomes

- **Available** → Applicant may proceed to registration.
- **Already registered** → Applicant is blocked.
- **Pending application exists** → Applicant is warned.
- **Invalid name format** → validation error shown.

7.3 Primary Workflow: New Street Name Registration

7.3.1 Actor

Applicant

7.3.2 Trigger

Applicant decides to register a new street name.

7.3.3 Steps

Step 1 — Start Application

1. Applicant clicks “Register Street Name”.
2. System creates application in `draft` state.

Step 2 — Fill Application Form

Applicant provides:

- proposed street name
- street type
- location details
- applicant details

System validates:

- required fields present
- street name format valid

Step 3 — Upload Documents

Applicant uploads:

- NIN slip
- passport photo
- royal recognition letter
- survey/property documents
- completed form

System validates:

- all required documents uploaded
- correct file formats

Step 4 — Submit Application

1. Applicant submits application.
2. System updates status → `awaiting_stage_a_payment`.

Step 5 — Submit Stage A Payment

1. Applicant enters payment reference/details.
2. System stores payment record.
3. Status → `stage_a_paid`.

Step 6 — Finance Confirmation

1. Finance reviews payment.
2. If valid:
   - status → `stage_a_confirmed`
3. Else:
   - status remains unchanged
   - rejection reason recorded

Step 7 — Naming Committee Validation

1. Naming Committee reviews recognition letter.
2. If valid:
   - status → `under_committee_review`
3. If invalid:
   - status → `naming_committee_rejected`

Step 8 — Committee Chairman Decision

1. Chairman reviews application.
2. Decision options:
   - Reject → `committee_rejected`
   - Approve → `approved_awaiting_stage_c_payment`

Step 9 — Stage C Payment

1. Applicant submits final payment.
2. Status → `stage_c_paid`

Step 10 — Finance Final Confirmation

1. Finance confirms final payment.
2. Status → `stage_c_confirmed`

Step 11 — Operational Progress Updates

Naming Committee updates sequential states:

- `signpost_pending`
- `maps_update_pending`
- `certificate_ready`

Step 12 — Completion

Final status:

- `completed`

7.3.4 Success Outcome

- Application fully processed
- Street name officially registered

7.3.5 Failure Scenarios

- Missing documents → blocked at submission
- Invalid payment → blocked at finance stage
- Invalid recognition letter → rejected
- Chairman rejection → final rejection

7.4 Workflow: Renewal of Street Name

7.4.1 Actor

Applicant

7.4.2 Trigger

Applicant wants to renew an approved street name.

7.4.3 Steps

1. Applicant selects existing approved record.
2. Clicks “Renew”.
3. System creates new renewal application.
4. Applicant submits renewal payment.
5. Finance confirms payment.

6. Status → `renewed`

7.4.4 Outcome

- Renewal linked to original street record
- Renewal tracked separately

7.5 Workflow: Payment Confirmation

7.5.1 Actor

Finance

7.5.2 Steps

1. Finance views payment queue.
2. Selects payment.
3. Reviews:
   - reference
   - amount
   - applicant
4. Confirms or rejects.

7.5.3 Outcomes

- Confirm → application progresses
- Reject → application remains blocked

7.6 Workflow: Document Validation

7.6.1 Actor

Naming Committee

7.6.2 Steps

1. View pending applications.
2. Open uploaded documents.
3. Validate recognition letter.

4. Record decision.

7.6.3 Outcomes

- Valid → proceed to chairman
- Invalid → reject

7.7 Workflow: Final Approval Decision

7.7.1 Actor

Committee Chairman

7.7.2 Steps

1. View screened applications.
2. Review:
   - documents
   - payment status
   - validation results
3. Make decision.

7.7.3 Outcomes

- Approve → Stage C payment triggered
- Reject → application closed

7.8 Workflow: Application Tracking

7.8.1 Actor

Applicant

7.8.2 Steps

1. Applicant logs in.
2. Navigates to dashboard.
3. Views application list.
4. Opens application detail.

7.8.3 Displayed Information

- current stage
- current status
- payment status
- validation results
- activity timeline

7.9 Workflow: Notification Handling

7.9.1 Trigger Events

Notifications shall be generated on:

- application submission
- payment submission
- payment confirmation
- validation result
- approval/rejection
- stage transitions

7.9.2 Actor

Applicant / Staff

7.9.3 Behavior

- notifications stored in system
- users can view notification history

7.10 Workflow: Staff Account Creation

7.10.1 Actor

Committee Chairman

7.10.2 Steps

1. Chairman opens admin interface.

2. Creates user account.
3. Assigns role:
   - Finance
   - Naming Committee
   - Chairman

7.10.3 Constraints

- No other role can create staff accounts
- Public registration cannot create staff roles

7.11 Cross-Workflow Rules

The following rules apply across all workflows:

- No workflow step shall be skipped.
- Status must always reflect real workflow position.
- Invalid transitions must be blocked.
- All actions must be logged.
- All role permissions must be enforced.

7.12 Edge Cases and Exception Flows

The system must handle:

7.12.1 Duplicate Street Name Attempt

- system blocks submission

7.12.2 Payment Not Confirmed

- application does not progress

7.12.3 Missing Documents

- submission blocked

7.12.4 Rejected Application

- cannot proceed further
- may require new application

7.12.5 Partial Completion

- system maintains current stage
- user can resume

7.13 Summary

This section defines the full operational lifecycle of the system, including:

- applicant initiation,
- multi-stage validation,
- role-based decision-making,
- payment-controlled progression,
- and post-approval operational tracking.

8. Functional Requirements

8.1 Overview

This section defines the **functional behavior of the system** in terms of:

- inputs,
- processing logic,
- validation rules,
- outputs, and
- error handling.

All requirements defined here shall be **strictly enforced in the backend** and reflected in the frontend.

8.2 User Authentication and Account Management

8.2.1 User Registration

Actor: Applicant

Requirements:

- The system shall allow public users to register as Applicants.
- Required fields:
  - full name
  - email (must be unique)
  - phone number
  - password

Validation Rules:

- Email must be unique.
- Password must meet minimum security rules (length ≥ 8).
- Phone number must be valid format.

Output:

- User account created.
- Role automatically assigned as `Applicant`.

Errors:

- Duplicate email → reject registration.
- Invalid input → validation error.

8.2.2 Login

Actor: All users

Requirements:

- Users shall log in using email and password.

Output:

- Authenticated session/token created.

Errors:

- Invalid credentials → login rejected.

8.2.3 Password Recovery

Requirements:

- System shall allow password reset via email.

8.3 Street Name Confirmation

8.3.1 Input

- Proposed street name

8.3.2 Processing Logic

- Normalize input (case-insensitive).
- Check:
  - approved street names
  - pending applications

8.3.3 Output

- available
- already registered
- pending
- invalid

8.3.4 Validation Rules

- Empty input → reject
- Invalid characters → reject

8.4 Application Management

8.4.1 Create Application

Actor: Applicant

Input:

- street name
- street type
- location details

Processing:

- Create application in `draft`

Validation:

- street name required
- street type required

8.4.2 Submit Application

Requirements:

- All required fields must be filled.
- All required documents must be uploaded.

Processing:

- Status → `awaiting_stage_a_payment`

Errors:

- Missing data → reject submission

8.4.3 Edit Application

Constraints:

- Only allowed in `draft` state
- Not allowed after submission

8.4.4 Cancel Application

Rules:

- Applicant may cancel before approval
- Status → `cancelled`

8.5 Document Management

8.5.1 Upload Documents

Required Documents:

- NIN slip
- passport
- recognition letter

- survey/property docs
- application form

Validation:

- All required documents must exist before submission
- File type restrictions (PDF/JPG/PNG)

Errors:

- Missing documents → block progression

8.5.2 Document Validation

Actor: Naming Committee

Processing:

- Mark document as:
  - valid
  - invalid

Rules:

- Recognition letter must be valid before proceeding

8.6 Payment Management

8.6.1 Submit Payment

Actor: Applicant

Input:

- payment reference
- amount
- payment stage (A or C)

Processing:

- Create payment record

Status:

- `stage_a_paid` or `stage_c_paid`

8.6.2 Confirm Payment

Actor: Finance

Processing:

- Validate payment record
- Mark as confirmed

Status Transition:

- Stage A → `stage_a_confirmed`
- Stage C → `stage_c_confirmed`

Errors:

- Invalid payment → reject confirmation

8.6.3 Payment Constraints

- Application cannot proceed without payment confirmation.
- Stage C payment cannot occur before approval.

8.7 Workflow State Transitions

8.7.1 General Rules

- All transitions must be validated.
- Invalid transitions must be blocked.
- Status must always reflect real workflow stage.

8.7.2 Valid Transitions (Simplified)

- draft → awaiting_stage_a_payment
- awaiting_stage_a_payment → stage_a_paid
- stage_a_paid → stage_a_confirmed
- stage_a_confirmed → under_naming_committee_review
- under_naming_committee_review → under_committee_review
- under_committee_review → approved_awaiting_stage_c_payment
- approved_awaiting_stage_c_payment → stage_c_paid

- stage_c_paid → stage_c_confirmed
- stage_c_confirmed → signpost_pending → maps_update_pending → certificate_ready → completed

---

8.7.3 Invalid Transitions

The system shall block:

- skipping stages
- reversing final states
- approval without validation
- payment bypass

8.8 Approval and Rejection

8.8.1 Naming Committee Rejection

- Status → `naming_committee_rejected`
- Application stops progression

8.8.2 Committee Chairman Rejection

- Status → `committee_rejected`
- Application closed

8.8.3 Approval

- Status → `approved_awaiting_stage_c_payment`

8.9 Renewal Management

8.9.1 Create Renewal

Actor: Applicant

Input:

- existing street record

Processing:

- Create new linked renewal application

8.9.2 Renewal Completion

- Payment required
- Status → `renewed`

8.10 Notifications

8.10.1 Trigger Events

- submission
- payment
- validation
- approval/rejection
- stage changes

8.10.2 Requirements

- Store notifications in database
- Display in user dashboard

8.11 Activity Logging

8.11.1 Requirements

The system shall log:

- application creation
- payment submission
- payment confirmation
- document validation
- approvals/rejections
- stage updates

8.11.2 Log Fields

- user
- action
- timestamp
- entity reference

8.12 Dashboard Requirements

8.12.1 Applicant Dashboard

- list of applications
- status tracking
- notifications

8.12.2 Finance Dashboard

- payment queue
- payment confirmations

8.12.3 Naming Committee Dashboard

- document validation queue
- operational stage updates

8.12.4 Committee Chairman Dashboard

- application pipeline
- approval queue
- financial summaries
- user management

8.13 Validation Rules (Global)

The system shall enforce:

- required fields must not be empty

- numeric fields must be valid
- file uploads must meet allowed types
- state transitions must be valid
- role permissions must be enforced

8.14 Error Handling

The system shall:

- return structured error responses
- prevent partial state corruption
- log all failures

8.15 Summary

The system shall implement:

- strict validation,
- deterministic workflow progression,
- role-based enforcement,
- complete audit logging,
- and complete data integrity across all actions.

No functional behavior shall be left undefined or assumed.

9. Detailed Module Specifications

9.1 Overview

The system shall be organized into clearly defined modules across:

- Frontend (Vue.js)
- Backend (Django + DRF)
- Data Layer (PostgreSQL via Django ORM)

Each module shall have:

- a single clear responsibility,

- defined inputs and outputs,
- no overlap in responsibilities, and
- clean interaction boundaries.

9.2 Frontend Module Specification (Vue.js)

9.2.1 Overview

The frontend shall be implemented as a **Vue 3 single-page application (SPA)** using:

- Vue Router for navigation
- Pinia for state management
- Mobile-first responsive layout

9.2.2 Frontend Module Structure

The frontend shall be organized into the following modules:

A. Authentication Module

Handles:

- login
- registration
- password recovery
- session management

B. Applicant Module

Handles:

- dashboard
- application creation
- application tracking
- renewal flow
- notifications

C. Finance Module

Handles:

- payment dashboard
- payment confirmation actions

D. Naming Committee Module

Handles:

- document validation interface
- operational stage updates (Stage D–F)

E. Committee Chairman Module

Handles:

- approval dashboard
- application review
- user management
- system overview dashboards

F. Shared UI Module

Reusable components:

- forms
- tables
- modals
- alerts
- loaders
- pagination

9.2.3 State Management (Pinia)

The frontend shall maintain structured state stores for:

- authStore (user + role)
- applicationStore
- paymentStore
- notificationStore
- adminStore (for chairman actions)

9.2.4 API Interaction Layer

The frontend shall include:

- a centralized API client
- request/response interceptors
- error handling standardization

9.2.5 Routing Structure

Routes must be role-based:

- `/login`
- `/register`
- `/dashboard` (role-dependent)
- `/applications`
- `/applications/:id`
- `/payments`
- `/admin/users`

Unauthorized access must be blocked.

9.3 Backend Module Specification (Django)

9.3.1 Overview

The backend shall be implemented using Django and Django REST Framework and organized into logical apps.

9.3.2 Backend App Structure

The backend shall be divided into the following Django apps:

A. `accounts` App

Handles:

- user model
- roles

- authentication
- user creation (Chairman-controlled for staff)

B. `applications` App

Handles:

- application creation
- application lifecycle
- status transitions
- renewal logic

C. `documents` App

Handles:

- document uploads
- file storage
- document validation flags

D. `payments` App

Handles:

- payment records
- payment stages (A, C)
- finance confirmation

E. `workflow` App

Handles:

- stage transitions
- validation of progression rules
- enforcement of workflow logic

F. `notifications` App

Handles:

- notification creation
- notification storage
- retrieval for users

G. `audit` App

Handles:

- activity logging
- tracking all key actions

H. `config` App

Handles:

- street types
- fee configuration
- system-level settings

9.3.3 Backend Layering

Each app must follow structured layers:

- models (data definitions)
- serializers (input/output validation)
- views (API endpoints)
- services (business logic)
- permissions (access control)

9.3.4 Business Logic Rules

- No business logic shall be placed only in views.
- Core logic must exist in service layer.
- State transitions must be centralized.

9.4 Authentication and Authorization Module

9.4.1 Responsibilities

- user authentication
- role assignment
- permission enforcement

9.4.2 Rules

- Role must be attached to user
- Permissions must be validated on every protected endpoint
- JWT or session must be consistent across frontend/backend

9.5 Application Module

9.5.1 Responsibilities

- create applications
- manage lifecycle
- enforce state transitions

9.5.2 Constraints

- Application must always have:
  - status
  - stage
- Invalid transitions must be blocked

9.6 Document Module

9.6.1 Responsibilities

- upload documents
- link to application
- store metadata
- validation status tracking

9.6.2 Constraints

- Documents must be required before submission

- Recognition letter must be validated

9.7 Payment Module

9.7.1 Responsibilities

- record payments
- handle Stage A and Stage C
- confirm payments

9.7.2 Constraints

- Payment must exist before confirmation
- Confirmation triggers state transition

9.8 Workflow Module

9.8.1 Responsibilities

- enforce state transitions
- validate workflow rules

9.8.2 Rules

- All transitions must pass through workflow logic
- No direct status updates allowed outside workflow module

9.9 Notification Module

9.9.1 Responsibilities

- create notifications
- store history
- serve notifications to users

9.10 Audit Module

9.10.1 Responsibilities

- log all key actions

9.10.2 Required Events

- login
- application creation
- payment submission
- validation
- approval/rejection

9.11 Config Module

9.11.1 Responsibilities

- manage:
  - street types
  - fees

9.11.2 Constraints

- Config must be dynamic
- No hardcoding in frontend

9.12 Inter-Module Communication

Modules shall interact through:

- service calls (backend)
- API calls (frontend → backend)

Direct cross-module data manipulation must be avoided.

9.13 Data Access Layer

- All DB interactions must go through Django ORM
- No raw SQL unless absolutely necessary
- Relationships must be enforced via models

9.14 Error Handling Across Modules

- All modules must return consistent error formats
- Errors must not expose sensitive data

9.15 Scalability Considerations

The architecture shall allow:

- addition of new roles
- addition of new stages
- payment gateway integration
- cloud storage migration

9.16 Constraints

- No module duplication
- No mixed responsibilities
- No bypassing workflow module
- No frontend-only logic for critical processes

9.17 Summary

The system shall be built as a modular, layered architecture with:

- clear separation of concerns,
- centralized workflow enforcement,
- reusable frontend components,
- scalable backend structure,
- and maintainable configuration-driven behavior.

---

10. UI/UX Specification

10.1 Overview

The system shall provide a **mobile-first responsive web interface** that supports all defined workflows for:

- Applicants,
- Finance users,
- Naming Committee users, and
- Committee Chairman users.

The interface shall:

- prioritize usability on small screens,
- scale cleanly to tablet and desktop,
- enforce role-based visibility of features, and
- clearly communicate application status and progression.

10.2 Design Principles

The UI shall follow these principles:

- **Clarity**: All actions and statuses must be understandable.
- **Consistency**: Reusable components and patterns must be used.
- **Guided workflows**: Users should always know the next step.
- **Feedback visibility**: Every action must produce visible feedback.
- **Mobile-first design**: Layout optimized for small screens first.

10.3 Information Architecture

The system shall be structured into the following top-level navigation areas:

Public (Unauthenticated)

- Home
- Confirm Street Name
- Login
- Register

Applicant Area

- Dashboard
- Applications
- Application Details
- Renewal
- Notifications
- Profile

Finance Area

- Dashboard
- Payments Queue
- Payment Details

Naming Committee Area

- Dashboard
- Validation Queue
- Application Details
- Operational Updates

Committee Chairman Area

- Dashboard
- Approval Queue
- Application Details
- User Management
- System Overview

10.4 Global UI Components

The system shall include reusable UI components:

- Navigation bar
- Sidebar (for desktop)
- Bottom navigation (for mobile)
- Data tables

- Forms
- Buttons
- Alerts (success, error, warning)
- Modals
- File upload components
- Status badges
- Loaders/skeleton screens
- Pagination controls

10.5 Page Specifications

10.5.1 Home Page (Public)

Purpose

- Entry point for users

Components

- system overview
- “Confirm Street Name” search
- login/register buttons

10.5.2 Confirm Street Name Page

Components

- input field for street name
- submit button
- result display panel

Behavior

- shows:
  - available
  - already registered
  - pending

10.5.3 Login Page

Fields

- email
- password

Actions

- login
- forgot password

10.5.4 Registration Page

Fields

- full name
- email
- phone
- password

10.5.5 Applicant Dashboard

Components

- summary cards:
  - total applications
  - pending
  - approved
- list of applications
- “New Application” button

10.5.6 Application List Page

Components

- table/list of applications
- filters:
  - status

    - stage
- search

10.5.7 Application Detail Page

Sections

A. Basic Info

- street name
- street type
- location

B. Status Tracker

- visual stepper:
    - Stage A → Stage F

C. Documents

- uploaded files
- download/view links

D. Payments

- Stage A and Stage C details

E. Activity Timeline

- chronological log of events

F. Actions

- submit payment
- upload documents (if pending)
- renewal button (if eligible)

10.5.8 New Application Page

Structure

Multi-step form:

Step 1

- street details

Step 2

- applicant details

Step 3

- document upload

Step 4

- review and submit

Behavior

- progress indicator
- validation per step

10.5.9 Payment Submission Page

Fields

- payment reference
- amount (pre-filled or validated)

Behavior

- submission updates status

10.5.10 Renewal Page

Components

- list of approved records
- renewal action button

10.5.11 Notifications Page

Components

- list of notifications
- timestamp
- status

10.5.12 Finance Dashboard

Components

- summary cards
- payment queue table

10.5.13 Payment Details Page

Components

- applicant info
- payment info
- confirm/reject buttons

10.5.14 Naming Committee Dashboard

Components

- validation queue
- pending documents

10.5.15 Document Validation Page

Components

- document viewer
- approve/reject buttons
- comment field

10.5.16 Operational Update Page

Components

- stage update controls
- status update buttons

10.5.17 Committee Chairman Dashboard

Components

- application counts
- approval queue
- financial summary

10.5.18 Approval Page

Components

- full application view
- approve/reject buttons
- decision notes

10.5.19 User Management Page

Components

- create user form
- role selection dropdown
- user list

10.6 Form Behavior

- All forms must validate before submission
- Inline validation errors must be shown
- Required fields must be clearly marked

10.7 Tables and Lists

- Support:
  - pagination
  - filtering
  - sorting
- Mobile:
  - switch to card layout

10.8 Status Representation

Status must be displayed using:

- color-coded badges
- readable labels

Example:

- Pending → yellow
- Approved → green
- Rejected → red

10.9 Loading States

- skeleton loaders for pages
- spinner for actions

10.10 Empty States

- show message:

    - “No applications yet”
- include action buttons

10.11 Error States

- show clear error message
- allow retry

10.12 Responsive Behavior

Mobile

- single-column layout
- bottom navigation

Tablet/Desktop

- multi-column layout
- sidebar navigation

10.13 Accessibility Requirements

- readable font sizes
- proper contrast
- keyboard navigability (basic)

10.14 Navigation Rules

- role-based routing
- unauthorized pages blocked
- redirect on login based on role

10.15 UX Feedback Requirements

Every user action must:

- show success message on completion
- show error message on failure

10.16 Constraints

- UI must not expose unauthorized actions
- UI must not allow skipping workflow steps
- UI must reflect backend state exactly

10.17 Summary

The UI shall:

- clearly represent workflow progression,
- guide users through tasks,
- adapt based on roles,
- enforce usability across devices,
- and provide full visibility into application state.

11. API Specification

11.1 Overview

The backend shall expose a **RESTful API** implemented using Django REST Framework.

The API shall:

- support all workflows defined in Section 7,
- enforce validation rules defined in Section 8,
- respect role-based permissions defined in Section 6,
- and provide structured, predictable request and response formats.

11.2 API Conventions

11.2.1 Base URL

/api/v1/

11.2.2 Content Type

All requests and responses shall use:

Content-Type: application/json

File uploads shall use:

multipart/form-data

11.2.3 Authentication

All protected endpoints shall require authentication via:

- session or token (JWT)

11.2.4 Standard Response Format

Success Response

```
{
  "success": true,
  "data": {...},
  "message": "Optional message"
}
```

Error Response

```
{
  "success": false,
  "error": {
```

```
    "code": "ERROR_CODE",
    "message": "Human readable message"
  }
}
```

11.2.5 HTTP Status Codes

- 200 → success
- 201 → created
- 400 → validation error
- 401 → unauthenticated
- 403 → unauthorized
- 404 → not found
- 500 → server error

11.3 Authentication Endpoints

11.3.1 Register Applicant

POST `/auth/register`

Request

```
{
  "full_name": "John Doe",
  "email": "john@example.com",
  "phone": "08012345678",
  "password": "password123"
}
```

Response

```
{
```

```
  "success": true,
  "data": {
    "user_id": "uuid",
    "role": "Applicant"
  }
}
```

11.3.2 Login

POST `/auth/login`

Request

```
{
  "email": "john@example.com",
  "password": "password123"
}
```

Response

```
{
  "success": true,
  "data": {
    "token": "jwt-token",
    "role": "Applicant"
  }
}
```

11.3.3 Password Reset

POST `/auth/password-reset`

11.4 Street Name Confirmation

11.4.1 Check Availability

GET `/street-names/check?name=example`

Response

```
{
  "success": true,
  "data": {
    "status": "available | registered | pending"
  }
}
```

11.5 Application Endpoints

11.5.1 Create Application

POST `/applications`

Request

```
{
  "street_name": "Example Street",
  "street_type": "Street",
  "location": "Ibeju-Lekki"
}
```

Response

```
{
  "success": true,
  "data": {
    "application_id": "uuid",
    "status": "draft"
  }
}
```

11.5.2 Submit Application

POST `/applications/{id}/submit`

Behavior

- validates required fields
- checks documents
- updates status

11.5.3 Get Applications (List)

GET `/applications`

Query Params

- status
- stage

11.5.4 Get Application Details

GET `/applications/{id}`

11.5.5 Cancel Application

POST `/applications/{id}/cancel`

11.6 Document Endpoints

11.6.1 Upload Document

POST `/applications/{id}/documents`

Content-Type: multipart/form-data

Fields

- file
- document_type

11.6.2 Get Documents

GET `/applications/{id}/documents`

11.6.3 Validate Document (Naming Committee)

POST `/documents/{id}/validate`

Request

```
{
  "status": "valid | invalid",
  "comment": "optional"
}
```

11.7 Payment Endpoints

11.7.1 Submit Payment

POST `/applications/{id}/payments`

Request

```
{
  "payment_reference": "ABC123",
  "amount": 5000,
  "stage": "A"
}
```

11.7.2 Get Payments

GET `/payments`

11.7.3 Confirm Payment (Finance)

POST `/payments/{id}/confirm`

11.7.4 Reject Payment

POST `/payments/{id}/reject`

11.8 Workflow Endpoints

11.8.1 Update Stage

POST `/applications/{id}/stage`

Request

```
{
  "stage": "signpost_pending"
}
```

11.8.2 Approve Application

POST `/applications/{id}/approve`

11.8.3 Reject Application

POST `/applications/{id}/reject`

11.9 Renewal Endpoints

11.9.1 Create Renewal

POST `/applications/{id}/renew`

11.10 Notification Endpoints

11.10.1 Get Notifications

GET `/notifications`

11.11 User Management Endpoints

11.11.1 Create Staff User (Chairman Only)

POST `/users`

Request

```
{
  "full_name": "Admin User",
  "email": "admin@example.com",
  "role": "Finance"
```

}

11.11.2 Get Users

GET `/users`

11.12 Filtering, Pagination, Sorting

11.12.1 Pagination

- `page`
- `page_size`

11.12.2 Filtering

- status
- stage
- role (for users)

11.13 Security Rules

- All endpoints must enforce role permissions
- Sensitive endpoints must validate ownership
- No direct DB exposure

11.14 Error Handling Rules

- All validation errors must return 400
- Unauthorized actions must return 403
- Missing resources must return 404

11.15 Idempotency Rules

- Payment confirmation must be idempotent
- Application submission must not duplicate

11.16 Constraints

- No undocumented endpoints
- No inconsistent response formats
- No bypassing workflow rules

11.17 Summary

The API shall:

- fully represent system functionality,
- enforce all business rules,
- maintain consistency,
- and support frontend and future integrations.

12. Data Specification

12.1 Overview

This section defines all **core data entities**, their fields, relationships, and constraints.

The system shall use a **relational data model** implemented in PostgreSQL via Django ORM.

All entities must:

- have clear ownership,
- enforce data integrity,
- support auditability,
- align with workflow and API specifications.

12.2 Core Entities

The system shall include the following core entities:

1. User
2. Role
3. Application
4. StreetType
5. Payment
6. Document
7. Notification
8. ActivityLog
9. Renewal
10. StageHistory

12.3 User Entity

12.3.1 Purpose

Represents all system users.

12.3.2 Fields

- id (UUID, primary key)
- full_name (string, required)
- email (string, unique, required)
- phone (string, required)
- password (hashed, required)
- role_id (foreign key → Role)
- is_active (boolean, default true)
- created_at (timestamp)
- updated_at (timestamp)

12.3.3 Constraints

- Email must be unique
- Role must exist
- Password must be hashed

12.4 Role Entity

12.4.1 Fields

- id (UUID)
- name (string)

12.4.2 Allowed Values

- Applicant
- Finance
- Naming Committee
- Committee Chairman

12.5 Application Entity

12.5.1 Purpose

Represents a street-name request or renewal.

12.5.2 Fields

- id (UUID)
- applicant_id (FK → User)
- street_name (string, required)
- street_type_id (FK → StreetType)
- location (string)
- status (enum)
- stage (enum)
- is_renewal (boolean, default false)
- parent_application_id (FK → Application, nullable)
- created_at
- updated_at

12.5.3 Constraints

- street_name must not duplicate approved names
- stage must align with status
- applicant_id required

12.6 StreetType Entity

12.6.1 Fields

- id (UUID)
- name (string)
- description (string)
- abbreviation (string)
- fee_amount (decimal)
- is_active (boolean)

12.6.2 Constraints

- name must be unique
- fee_amount must be ≥ 0

12.7 Payment Entity

12.7.1 Fields

- id (UUID)
- application_id (FK → Application)
- payer_id (FK → User)
- payment_reference (string)
- stage (enum: A, C)
- amount (decimal)
- status (enum: pending, confirmed, rejected)
- confirmed_by (FK → User, nullable)
- confirmed_at (timestamp, nullable)
- created_at

12.7.2 Constraints

- payment_reference must be unique
- amount must be positive
- stage must match application progression

12.8 Document Entity

12.8.1 Fields

- id (UUID)
- application_id (FK → Application)

- document_type (enum)
- file_path (string)
- status (enum: pending, valid, invalid)
- validated_by (FK → User, nullable)
- validated_at (timestamp, nullable)
- created_at

12.8.2 Document Types

- NIN
- PASSPORT
- RECOGNITION_LETTER
- SURVEY_DOC
- APPLICATION_FORM

12.8.3 Constraints

- All required document types must exist before submission
- Recognition letter must be validated

12.9 Notification Entity

12.9.1 Fields

- id (UUID)
- user_id (FK → User)
- message (string)
- read (boolean, default false)
- created_at

12.10 ActivityLog Entity

12.10.1 Fields

- id (UUID)
- user_id (FK → User)
- action (string)
- entity_type (string)
- entity_id (UUID)

- metadata (JSON, optional)
- created_at

12.10.2 Purpose

Tracks:

- approvals
- payments
- document validation
- status changes

12.11 Renewal Entity

12.11.1 Fields

- id (UUID)
- application_id (FK → Application)
- original_application_id (FK → Application)
- created_at

12.11.2 Constraints

- original must be approved

12.12 StageHistory Entity

12.12.1 Fields

- id (UUID)
- application_id (FK → Application)
- previous_stage (string)
- new_stage (string)
- changed_by (FK → User)
- created_at

12.12.2 Purpose

Tracks all stage transitions for auditability.

12.13 Enumerations

12.13.1 Application Status

- draft
- submitted
- awaiting_stage_a_payment
- stage_a_paid
- stage_a_confirmed
- under_naming_committee_review
- naming_committee_rejected
- under_committee_review
- committee_rejected
- approved_awaiting_stage_c_payment
- stage_c_paid
- stage_c_confirmed
- signpost_pending
- maps_update_pending
- certificate_ready
- completed
- renewal_pending
- renewed
- cancelled

12.13.2 Application Stage

- Stage A
- Stage B
- Stage C
- Stage D
- Stage E
- Stage F

12.13.3 Payment Status

- pending
- confirmed
- rejected

12.13.4 Document Status

- pending
- valid
- invalid

12.14 Relationships

- User → Role (Many-to-One)
- Application → User (Many-to-One)
- Application → StreetType (Many-to-One)
- Application → Payment (One-to-Many)
- Application → Document (One-to-Many)
- Application → ActivityLog (One-to-Many)
- Application → StageHistory (One-to-Many)
- Renewal → Application (Many-to-One)

12.15 Data Integrity Rules

The system shall enforce:

- no orphan records
- no invalid foreign keys
- no invalid status transitions
- no duplicate payment references
- no missing required documents

12.16 Soft Delete Strategy

- Entities may include `deleted_at`
- Deleted records must not appear in active queries

12.17 Indexing Requirements

The system shall index:

- user.email
- application.status

- application.stage
- payment.payment_reference
- document.application_id

12.18 Audit Requirements

Every important entity must include:

- created_at
- updated_at

Critical actions must be logged in ActivityLog.

12.19 Data Lifecycle Rules

- Applications persist permanently
- Payments must not be deleted
- Logs must be immutable
- Documents must remain linked to applications

12.20 Summary

The data model shall:

- support full workflow tracking,
- enforce data integrity,
- enable auditability,
- align with API and UI,
- and remain extensible for future enhancements.

Perfect—this is the final stretch.

13. Database Design

13.1 Overview

The system shall use **PostgreSQL** as the primary database and shall implement a **normalized relational schema** using Django ORM.

The database design shall enforce:

- data integrity,
- referential consistency,
- performance through indexing,
- and auditability of critical operations.

13.2 Primary Key Strategy

- All entities shall use **UUID (version 4)** as primary keys.
- UUIDs shall be generated at the application layer.

13.3 Foreign Key Constraints

- All foreign key relationships shall enforce referential integrity.
- Deletion rules:
  - `CASCADE` for dependent entities (e.g., documents under application)
  - `PROTECT` for critical references (e.g., payments, logs)

13.4 Unique Constraints

The system shall enforce:

- User.email → UNIQUE
- Payment.payment_reference → UNIQUE
- StreetType.name → UNIQUE
- Application (street_name + approved state) → UNIQUE

13.5 Composite Constraints

- Prevent duplicate active applications for same street:
  - `(street_name, status NOT IN [rejected, cancelled])`

13.6 Indexing Strategy

The system shall include indexes for performance:

High Priority Indexes

- User.email
- Application.applicant_id
- Application.status
- Application.stage
- Payment.payment_reference
- Payment.application_id
- Document.application_id

Optional Indexes

- ActivityLog.entity_id
- Notification.user_id

13.7 Transaction Management

All critical operations must be atomic:

Must use DB transactions for:

- application submission
- payment confirmation
- approval/rejection
- stage transitions

13.8 Concurrency Control

The system shall prevent race conditions:

- Payment confirmation must be idempotent
- Approval actions must use row-level locking
- Duplicate submissions must be blocked

13.9 Soft Delete Strategy

Where applicable:

- add `deleted_at` field
- exclude soft-deleted records from queries

Exceptions:

- Payments and ActivityLogs must never be deleted

13.10 Audit Data Integrity

- ActivityLog must be immutable
- No updates allowed after creation

13.11 Data Migration Rules

- All schema changes must be handled via migrations
- No direct DB schema edits allowed

13.12 Backup and Recovery Assumptions

- Database must support backup strategy
- Data loss must be minimized

13.13 Summary

The database shall:

- enforce integrity,
- prevent duplication,
- support audit,
- and scale with system growth.

---

22. Acceptance Criteria

22.1 Overview

The system shall only be considered complete if all acceptance criteria below are satisfied.

22.2 Global Acceptance Criteria

- Application runs locally without errors
- All APIs respond correctly
- UI renders across mobile and desktop
- Authentication works for all roles

22.3 Applicant Acceptance Criteria

- Applicant can:
  - register
  - login
  - create application
  - upload documents
  - submit payment
  - track application

22.4 Finance Acceptance Criteria

- Finance can:
  - view payment queue
  - confirm Stage A payment
  - confirm Stage C payment

22.5 Naming Committee Acceptance Criteria

- Naming Committee can:
  - view applications
  - validate recognition letter
  - update operational stages

22.6 Committee Chairman Acceptance Criteria

- Chairman can:
    - approve/reject applications
    - view dashboards
    - create staff accounts

22.7 Workflow Acceptance Criteria

- All stage transitions follow defined rules
- Invalid transitions are blocked
- Status always reflects correct stage

22.8 Data Integrity Acceptance Criteria

- No duplicate payment references
- No missing required documents
- No orphan records

22.9 API Acceptance Criteria

- All endpoints return correct responses
- Error handling works
- Role permissions enforced

22.10 UI Acceptance Criteria

- All pages load correctly
- Forms validate correctly
- Mobile responsiveness works

22.11 Test Coverage Criteria

- Core flows tested:
    - application submission

    - payment confirmation
    - approval flow

22.12 Completion Criteria

System is complete when:

- all features work end-to-end
- no critical bugs exist
- setup is documented

---

24. Implementation Guidance for Copilot

24.1 Repository Structure

The system shall be structured as:

/frontend

/backend

/docker

/docs

24.2 Backend Structure

/backend

  /accounts

  /applications

  /documents

  /payments

  /workflow

/notifications

/audit

/config

24.3 Frontend Structure

/frontend

/src

/modules

/components

/pages

/store

/services

24.4 Coding Rules

- Use clear naming conventions
- Avoid unnecessary abstraction
- Keep business logic in services
- No duplication of logic

24.5 File Creation Rules

Copilot must:

- create full working files
- not generate partial code
- not leave TODO comments

24.6 Environment Configuration

Must include:

- `.env` files
- database config
- secret keys

24.7 Docker Requirements

Must include:

- backend container
- frontend container
- postgres container

24.8 Implementation Order

Strictly follow:

1. Models
2. Migrations
3. Auth
4. APIs
5. Frontend
6. Tests
7. Docker

24.9 Output Requirements

Copilot must produce:

- runnable application
- clean folder structure
- working endpoints
- working UI

24.10 Stop Conditions

Copilot must stop only when:

- all endpoints implemented
- UI connected to backend
- no missing features

24.11 Forbidden Behavior

Copilot must not:

- invent new features
- skip validation
- ignore roles
- change stack

24.12 Summary

This section ensures:

- predictable output
- controlled architecture
- no hallucinated design decisions